Design a smart-city emergency routing system with 50+ intersections (Nodes) and 100+ weighted roads (Edges and Weights).

Submitted in the partial fulfilment for the Course

of

CSA0311- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B. TECH -AIDS,AIML

B.E CSE

Submitted by

M. Vaishali(192524440)

R. Sweta(192511408)

V. Deepa Darshini(192525213)

Under the Supervision of

Dr. K. Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CSA0311 Data Structure
Academic Year / Batch	2nd year (2025-2029)
Faculty Name	Dr.K.Kiruthika
Assignment Title	Design a smart-city emergency routing system with 50+intersections (Nodes) and 100 +weighted roads (Edges and Weights).
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data type for construction of the smart city emergency system CO4: Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C. CO5: Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall.
Bloom's Taxonomy Level	L4 – Analyze
SDG Mapping (SDG 1-17)	SDG-9 and SDG-11
Industry / Societal Relevance	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities;

B. Assignment Problem / Challenge

Design a Smart-City Emergency Vehicle Routing System with 50 intersections and 100 weighted roads that uses appropriate graph data structures and shortest-path algorithms to provide efficient real-time routes for ambulances and fire engines under dynamic traffic conditions. Compare alternative algorithms and justify the final engineering solution based on performance, memory usage, scalability, and route optimality

C. Problem Statement

Problem / Case / Design Challenge:

A smart-city traffic management company is developing a real-time emergency vehicle routing system for ambulances and fire engines that receives road-network information containing intersections, roads, distances, and current traffic conditions, and must operate under the following constraints: the road network may contain more than 100 intersections and 200 roads, the system must provide a route within 200 ms, emergency vehicles should be directed through the shortest available route, the system must support frequent updates to road conditions, memory usage must remain within moderate computational resources, and the system should support both one-to-one route queries and repeated route queries from multiple locations, with the engineering team considering different data structures and algorithms including adjacency matrices, adjacency lists, BFS, Dijkstra's algorithm, Floyd-Warshall, heaps/priority queues, and hash tables. The student develops a complete, C program based emergency vehicle routing system that integrates Dijkstra's algorithm with a Min-Heap priority queue for optimal weighted shortest-path computation, hash-based graph storage using HashMaps for adjacency lists and HashSets for O(1) visited-node tracking, a dynamic traffic management module with HashMap-based O(1) updates

D. Requirements and Constraints

- **Functional:** Find the shortest route between two intersections.
- **Performance:** Route calculation within **200 ms**.
- **Technical:** 50 nodes, 100 weighted edges; use **Dijkstra/A***.
- **Safety:** Avoid blocked or unavailable roads.
- **Reliability:** Provide accurate routes and handle traffic updates.
- **User:** Simple interface to select source and destination.
- **Security:** Protect vehicle/location data.
- **Cost:** Use low-cost/free software and tools.

E. Student Work

1. Problem Understanding and Formulation

1.1 What is the problem?

In a smart city, emergency vehicles such as ambulances and fire engines must reach their destinations as quickly as possible. Road conditions may change continuously because of traffic congestion, accidents, and road blockages. A fixed route may therefore become inefficient or unavailable.

The proposed Real-Time Emergency Vehicle Routing System models the city road network as a weighted graph. Intersections are represented as nodes and roads as edges. Each road has a physical distance and a traffic condition. The system calculates a traffic-adjusted cost and uses shortest-path algorithms to determine an efficient route.

1.2 Expected outcomes

Find the shortest available route between a source and destination.

Consider current traffic conditions.

Avoid blocked roads.

Support ambulance and fire-engine routing.

Dynamically update road traffic conditions.

Recalculate routes after traffic changes or incidents.

Display route distance, traffic cost, estimated travel time and nodes explored.

Visualize the selected route.

Compare Dijkstra, BFS and Floyd-Warshall algorithms.

1.3 Available information/data

Data	Description
Intersections	Represented by nodes such as A, B, C, etc.
Roads	Connections between intersections
Distance	Physical road distance in kilometres
Traffic Level	LOW, MEDIUM, HIGH, HEAVY or BLOCKED
Traffic Multiplier	Used to calculate traffic-adjusted cost
Vehicle Type	Ambulance or Fire Engine

1.4 Assumptions

1. Roads are represented as an undirected graph.
2. All road distances are positive.
3. Traffic conditions are represented using predefined levels.
4. Traffic conditions can change during operation.
5. Blocked roads cannot be used.
6. The shortest route is based on traffic-adjusted cost.
7. Emergency vehicle speed is assumed according to vehicle type.
8. The demonstration network is a simplified representation of a smart-city road network.

1.5 Constraints

- The network may contain more than 100 intersections and 200 roads.
- Route computation should support real-time use.
- The shortest available route must be selected.
- Traffic conditions must support frequent updates.
- Blocked roads must be avoided.
- Memory usage should remain moderate.

2. Application of Course Knowledge

2.1 Graph Data Structure

The city is represented as a graph:

G = (V, E)

where:

V = set of intersections

E = set of roads

Each road contains:

Distance

Traffic condition

Traffic-adjusted weight

The code stores these attributes for every road and retrieves neighboring intersections through the graph structure.

2.2 Dijkstra's Algorithm

Dijkstra's algorithm is the main routing algorithm.
The algorithm repeatedly selects the node having the smallest current cost and explores its neighboring roads.
The implementation:

- Starts from the source node.
- Inserts it into a priority queue.
- Removes the lowest-cost node.
- Checks its neighboring roads.
- Calculates the new cost.
- Updates the candidate route when a lower cost is found.
- Stops when the destination is reached.

The program uses heapq as the priority queue and stores cost, node, path and distance in each heap entry.

2.3 Min-Heap / Priority Queue

A Min-Heap is used to efficiently select the node with the lowest current routing cost.
The implementation uses:
heapq.heappush()
and
heapq.heappop()
This supports efficient priority-based node processing. The application also displays the priority queue state during its educational Dijkstra animation.

2.4 Visited Tracking

The Dijkstra implementation maintains a visited structure to prevent unnecessary reprocessing of nodes whose best-known cost has already been established.
The application also demonstrates the concept of HashSet-based visited tracking in its educational data-structure display.

2.5 BFS

BFS is included for algorithm comparison. It explores the graph level by level and is useful for unweighted shortest-path problems.
However, because emergency roads have different distances and traffic costs, BFS is not the primary algorithm for this application.

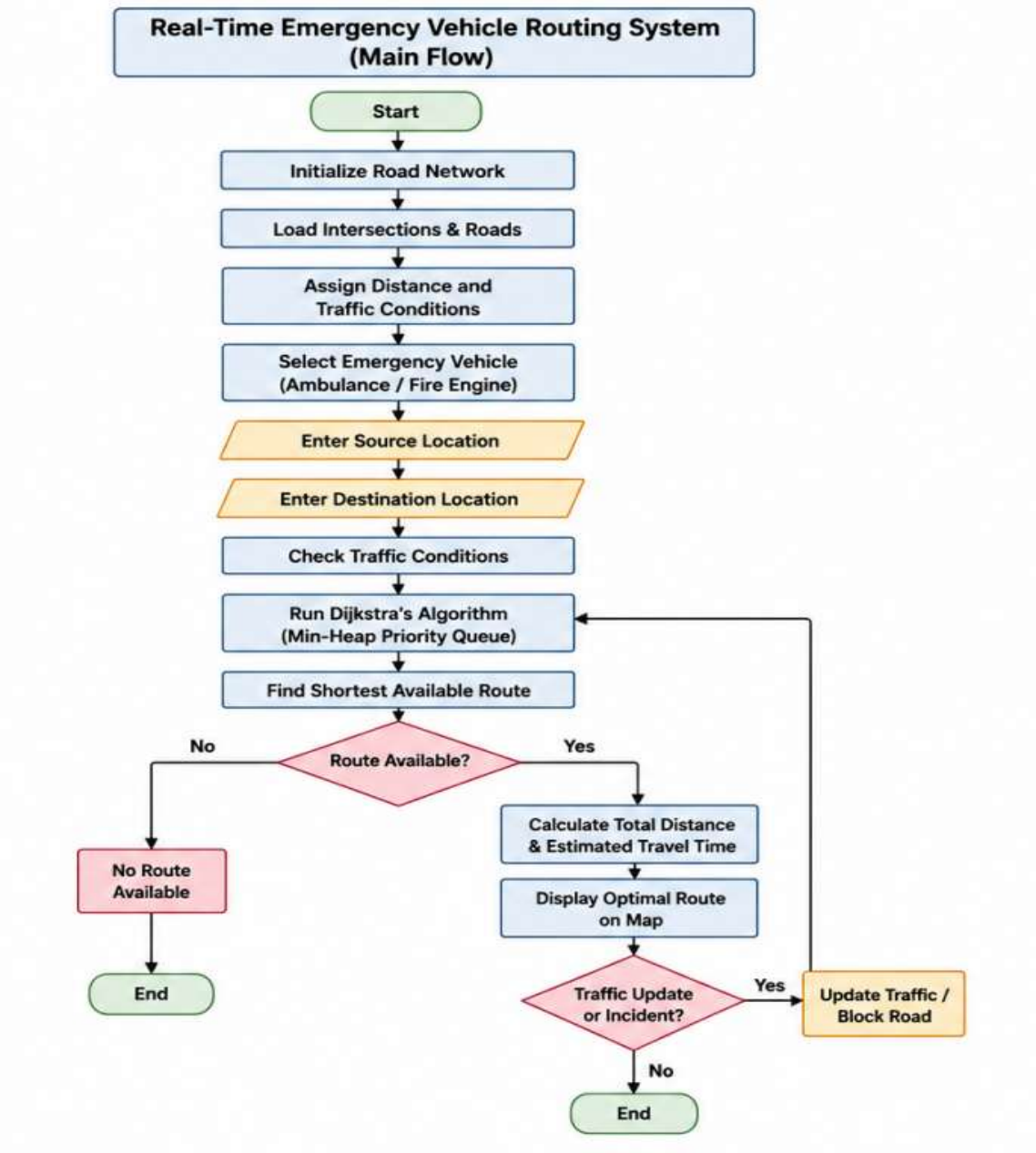
2.6 Floyd-Warshall

Floyd-Warshall is implemented as another comparison algorithm. It calculates shortest paths between all pairs of nodes using a distance matrix.
This provides a useful comparison with Dijkstra for repeated route-query scenarios.

2.9 Complexity

For Dijkstra with a binary Min-Heap, the expected complexity is approximately:
$O((V + E) \log V)$
The application itself displays this complexity as part of its educational data-structure section.

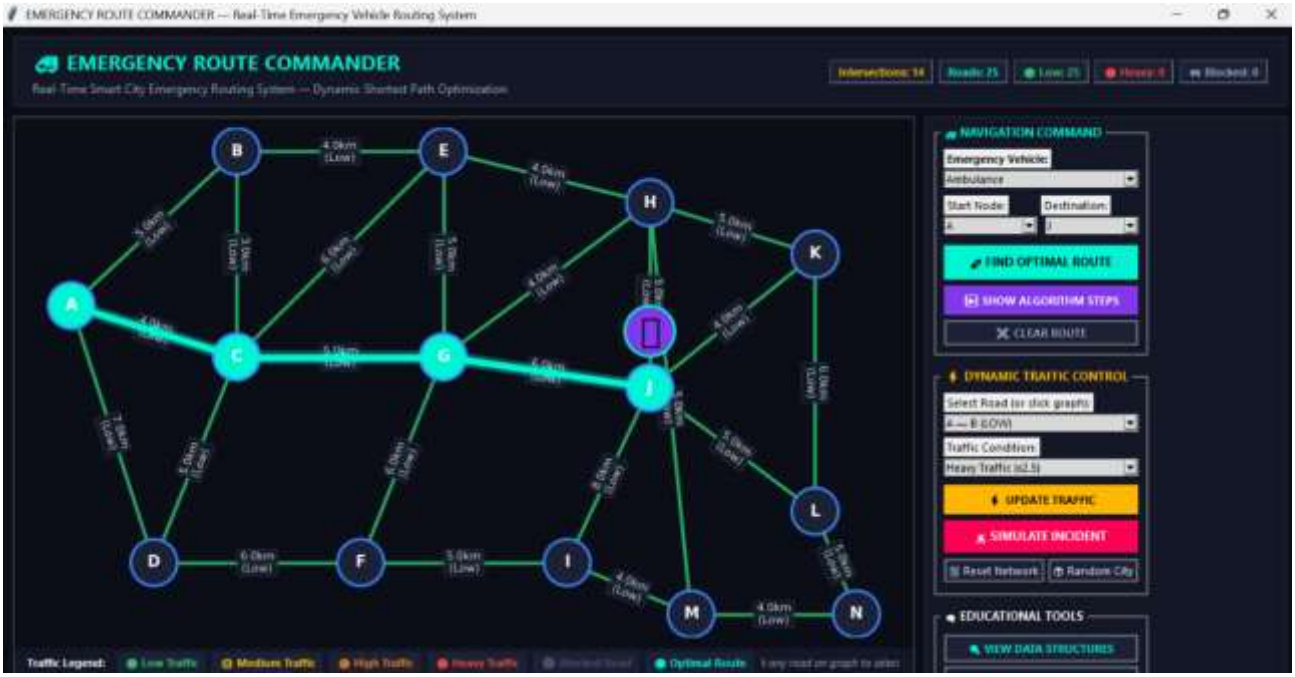
3. Solution / Design / Methodology



4. Use of Modern Tools

Tool/Library	Purpose
Python	Main programming language
Tkinter	GUI development
NetworkX	Graph creation and management
heapq	Min-Heap priority queue
Matplotlib	Graph visualization
time	Execution-time measurement
random	Random traffic/incident and city generation
math	Distance calculations for graph interaction

5. Results and Validation



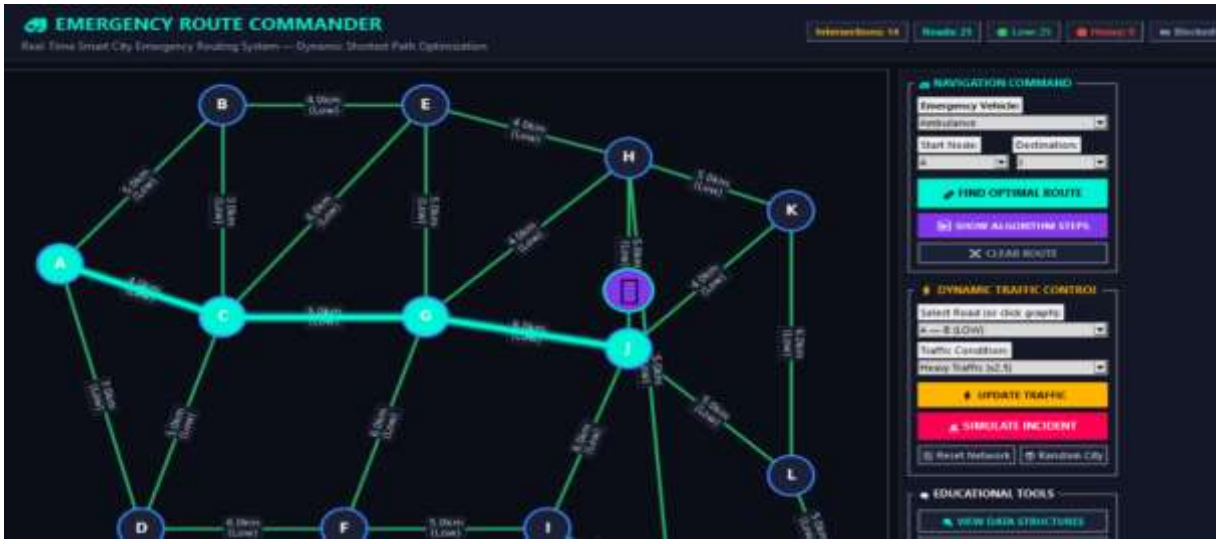
Source Code:

```
class TrafficGraph:
    def __init__(self):
        self.graph = nx.Graph()
        self.pos = {}
        self.node_names = {}
        self.load_default_smart_city()

    def add_intersection(self, node_id, label=None, pos=None):
        self.graph.add_node(node_id)
        self.node_names[node_id] = label if label else str(node_id)

        if pos:
            self.pos[node_id] = pos

    def add_road(self, u, v, distance_km, traffic_level='LOW'):
        level = traffic_level if traffic_level in TRAFFIC_LEVELS else 'LOW'
        mult = TRAFFIC_LEVELS[level]['multiplier']

        weight = distance_km * mult if mult != float('inf') else float('inf')

        self.graph.add_edge(
            u, v,
            distance=float(distance_km),
            traffic=level,
            weight=weight
        )
```

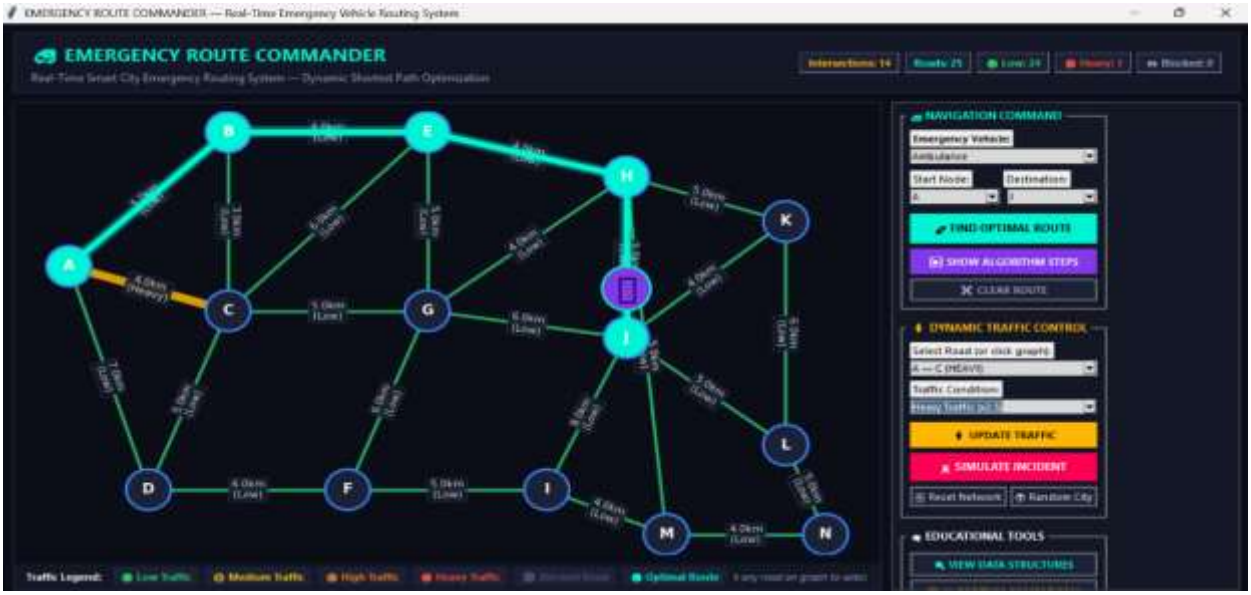
```
class EmergencyRouteCommanderApp:
    def __init__(self, root):
        self.root = root

        self.root.title(
            "EMERGENCY ROUTE COMMANDER"
        )

        self.root.geometry("1440x880")

        self.traffic_graph = TrafficGraph()
        self.last_route_result = None
        self.selected_edge = None

        self._configure_styles()
        self._build_header()
        self._build_main_layout()

        self._refresh_comboboxes()
        self.visualizer.draw_network(
            self.traffic_graph
        )
# Application Entry Point
if __name__ == '__main__':
    root = tk.Tk()
    app = EmergencyRouteCommanderApp(root)
    root.mainloop()
```

Test Cases:

- **Case 1: Standard Optimal Routing**
    - **Start/Destination:** Node A to Node J.
    - **Condition:** All surrounding roads operate under normal/low traffic conditions.
    - **Result:** The system computes the shortest baseline path directly through intermediate nodes C and G (A→C→G→J).



- **Case 2: Dynamic Traffic Congestion Re-routing**
    - **Start/Destination:** Node A to Node J.
    - **Condition:** The primary edge between Node A and Node C is subjected to heavy traffic (2.5× weight penalty).
    - **Result:** The routing engine dynamically recalculates and reroutes the emergency vehicle via a northern bypass (A→B→E→H→J) to avoid congestion delays.



- **Case 3: Incident Road Blockade Handling**
    - **Start/Destination:** Node A to Node J.
    - **Condition:** Roads connecting Node A to nodes B and C are completely blocked (∞ weight) due to simulated incidents.
    - **Result:** The algorithm adapts to total path failures by diverting the vehicle through the southern node network (A→D→C→G→J).

6. Analysis and Engineering Decision

Final Engineering Decision

Dijkstra + Min-Heap was selected as the primary routing solution because the road network contains weighted edges representing both distance and traffic conditions.

BFS is less suitable because it primarily considers the number of edges rather than their weighted cost.

Floyd-Warshall is useful when shortest paths between many pairs are required, but it requires an all-pairs computation and a matrix representation.

Therefore, Dijkstra with a Min-Heap provides a good balance between route optimality, computational efficiency and memory usage for the intended emergency routing application

Algorithm Comparision

Algorithm	Advantage	Limitation	Suitability
BFS	Simple and fast for unweighted graphs	Does not optimize weighted traffic cost	Low
Dijkstra + Min-Heap	Finds weighted shortest route efficiently	Requires non-negative weights	**High**
Floyd-Warshall	Useful for many source-destination queries	Higher memory and computation cost	Medium

7. Broader Considerations

7.1 Safety

The system can help emergency vehicles avoid congested and blocked roads, potentially reducing emergency response time.

7.2 Society

Faster routing can improve emergency-service accessibility for citizens and support ambulance and fire-service operations.

7.3 Environment

Efficient routing can reduce unnecessary travel distance and vehicle idling, potentially reducing fuel consumption and emissions.

7.4 Economics

Reducing emergency travel time can improve utilization of emergency vehicles and reduce operational inefficiencies.

7.5 Professional Responsibility

Because emergency routing is safety-critical, a real-world deployment would require reliable traffic data, extensive testing, fault handling and human oversight.

8. Conclusion

The Real-Time Emergency Vehicle Routing System successfully demonstrates how Data Structures and Algorithms can be applied to a smart-city emergency management problem. The system represents the road network as a weighted graph and uses traffic-adjusted road costs to identify an efficient emergency route.

Dijkstra's algorithm combined with a Min-Heap is used as the main shortest-path technique. The application also provides BFS and Floyd-Warshall for algorithm comparison. Dynamic traffic updating and incident simulation demonstrate how changing road conditions can influence the selected route. The GUI provides route visualization, traffic management, vehicle selection and performance metrics.

The project achieves its main objective of demonstrating dynamic weighted shortest-path routing for emergency vehicles. However, future versions could integrate real-time GPS/traffic APIs, larger road networks, live vehicle tracking and more advanced routing techniques.

9. Student Reflection

Through this assignment, I learned how theoretical concepts from Data Structures and Algorithms can be applied to a practical smart-city problem. I gained a better understanding of graph representation, weighted shortest-path algorithms, priority queues, traffic-based edge weights and dynamic graph updates.

I also learned that choosing an algorithm depends on the nature of the problem. BFS is useful for unweighted problems, while Dijkstra is more appropriate when roads have different costs. Implementing an algorithm comparison module helped me understand the differences between Dijkstra, BFS and Floyd-Warshall.

Another important learning outcome was understanding how a graphical user interface can be combined with algorithms to create an interactive engineering application.

applications.

10. References

[1] NetworkX Developers, NetworkX 3.3 Documentation, Apr. 2024. [Online]. Available: NetworkX Documentation

[2] NetworkX Developers, NetworkX 3.4 Documentation, Oct. 2024. [Online]. Available: NetworkX 3.4 Documentation

[3] Python Software Foundation, “heapq — Heap queue algorithm,” Python 3.14 Documentation, 2026. [Online]. Available: Python heapq Documentation

[4] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python 3.14 Documentation, 2026. [Online]. Available: Python Tkinter Documentation

[5] NetworkX Developers, “NetworkX: Graph types and algorithms,” NetworkX 3.4 Documentation, 2024. [Online]. Available: NetworkX Reference Documentation

[6] Python Software Foundation, “Python 3.12 Documentation,” Python Documentation, 2023. [Online]. Available: Python 3.12 Documentation

[7] Python Software Foundation, “Python 3.13 Documentation,” Python Documentation, 2024. [Online]. Available: Python 3.13 Documentation

[8] NetworkX Developers, “NetworkX algorithms,” NetworkX Documentation, 2024. [Online]. Available: NetworkX Algorithms

[9] Python Software Foundation, “Data structures — Lists, sets and dictionaries,” Python 3 Documentation, 2026. [Online]. Available: Python Data Structures Documentation

[10] Python Software Foundation, “Graphical user interface FAQ,” Python Documentation, 2026. [Online]. Available: Python GUI Documentation

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mappin

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action